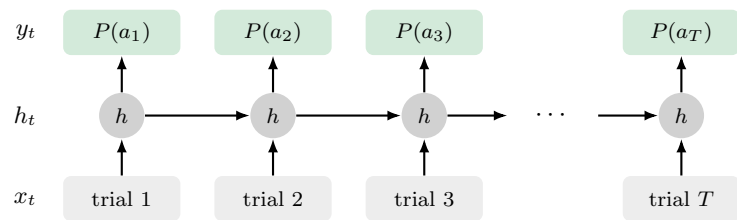


Many-to-many: every step gets an input x_t and produces an output y_t ; the loss is summed over the steps that carry a label

1. Trial-level tiny RNN (one step = one trial; the sequence is a whole session)



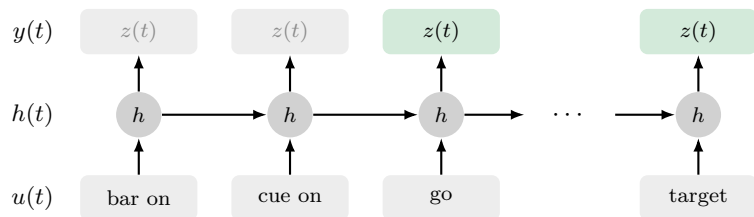
$y_t = P(a_t)$: probability of Short / Mid / Long on trial t compared with the animal's choice a_t (cross-entropy)

h_t : $d = 1-4$ variables carried from trial to trial

$x_t = (a_{t-1}, s_{t-1}, r_{t-1}, s_t, \text{cue}_t)$: previous choice, previous bar, previous outcome, current bar, rule

Every trial has a label, so every step contributes to the loss. Sequence length $T = 240-465$ trials.

2. Task network (one step = 10 ms; the sequence is one trial, ≈ 335 steps)



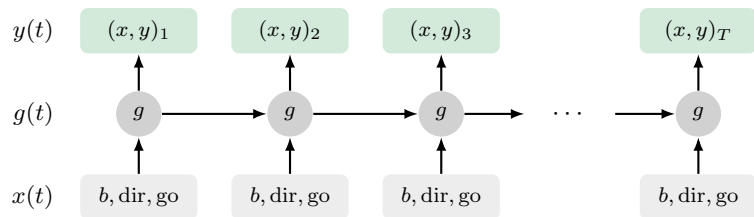
$y(t) = z(t)$: category read-out at every step, but the loss is **masked**: only steps after go (green) are compared with a_t

$h(t)$: N rate units; must hold the bar through the cue

$u(t) = (\text{bar}(t), \text{rule}(t), \text{go}(t))$: bar length while it is shown, 2-/3-cat channel from the cue, go from target onset

One label per trial, repeated over the post-go steps: this is where many-to-many looks like many-to-one. Keeping the per-step output is what gives the model's decision time.

3. Motor network (one step = 10 ms; same trial, aligned step by step)



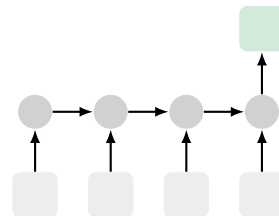
$y(t) = (x, y)(t)$: cursor position at every step, compared with the measured trajectory (MSE on valid steps)

$g(t)$: GRU state, M units

$x(t) = (b(t), \text{dir}, \text{go}(t))$: bottleneck from the task network, chosen direction one-hot, go

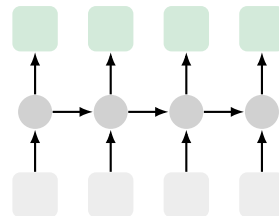
A measured position at every step, so every valid step contributes. This is the purest many-to-many of the three.

Many-to-one



reads the whole sequence, one label at the end (e.g. the section 1.10 decoder)

Many-to-many (aligned)



one output per step; a mask decides which steps enter the loss (rows 1-3)